Quantum Encoding Agents: A Natural Language Interface for Data Embedding Strategy Selection in Quantum Machine Learning

Ana Paula Appel
Red Hat, Inc.
aappel@redhat.com

Abstract

Selecting an appropriate data encoding strategy is one of the most consequential and least-tooled decisions in Quantum Machine Learning (QML) pipelines. The choice of quantum feature map determines the structure of the Hilbert space into which classical data is embedded, directly shaping the expressibility of quantum kernels, the trainability of variational circuits, and the feasibility of execution on near-term hardware. Despite its central role, encoding selection is rarely addressed systematically: practitioners typically default to a single strategy — most often angle encoding — without analyzing how the structural characteristics of their data interact with the properties of each encoding family. This paper presents **Quantum Encoding Agents**, an open-source system that reformulates encoding selection as a natural language interaction. Given a dataset and an optional description of the QML task, the system analyzes the data profile, applies a hardware-aware recommendation policy grounded in the critical gate error threshold $p^* \approx 10^{-3}$, generates a complete copyable Qiskit circuit, produces a natural language justification in the user's language (Portuguese or English), and computes the quantum kernel matrix with Kernel-Target Alignment scoring. The system is implemented as a FastAPI microservice, deployed on Red Hat OpenShift with NVIDIA OpenShell sandbox isolation, and exposed through OpenClaw agents with distinct epistemic personalities calibrated to different levels of user expertise. Evaluation confirms that the system correctly selects among seven encoding families — amplitude, angle, dense angle, IQP, basis, data re-uploading, and custom feature map — under realistic NISQ hardware constraints, and that KTA scores discriminate between encodings on benchmark datasets.

Keywords: Quantum Machine Learning, Quantum Feature Maps, Data Encoding, AI Agents, NISQ Hardware, OpenShift, Natural Language Interface.

---

1. Introduction

The practical adoption of Quantum Machine Learning faces a fundamental bottleneck that precedes model training: the problem of quantum data encoding. Before any quantum classifier, kernel method, or variational circuit can operate, classical data must be mapped into quantum states. This mapping — the encoding or feature map — is not a neutral preprocessing step. It defines the geometry of the state space, the structure of correlations captured between features, and the depth and qubit count of the resulting circuit. Different encodings impose radically different tradeoffs:

amplitude encoding compresses n features into $\lceil \log_2 n \rceil$ qubits but requires a state preparation circuit of exponential depth; angle encoding produces shallow circuits with one qubit per feature but limited expressibility; IQP encoding [2] captures cross-feature correlations through diagonal unitaries and has strong theoretical guarantees for quantum kernels, but at greater circuit depth.

Despite the critical nature of this choice, the QML literature offers little guidance on encoding selection as a function of data characteristics. Practitioners encounter a combinatorial decision problem: seven or more encoding families, each with different qubit efficiency, circuit depth, hardware sensitivity, and affinity for different QML algorithms, must be matched against a dataset whose relevant properties — dimensionality, value type, distribution, sign — interact with the encoding in non-obvious ways. The seminal survey by Sammartino [1], reviewing 66 papers from 2017 to 2026, identifies this as the most underaddressed problem in applied QML, formalizes six encoding families, and derives a critical gate error rate threshold $p^* \approx 10^{-3}$ above which deep encodings become impractical on NISQ hardware.

This paper addresses the encoding selection problem through a different lens: rather than proposing a new encoding or a new theoretical framework, I build a practical agent system that makes existing knowledge actionable. I make the following contributions:

1. **A hardware-aware encoding recommendation engine** that applies the p^* threshold from [1], accounting for gate error rate, circuit depth budget, qubit count, and chip connectivity topology.

2. **Seven implemented encoding strategies** in Qiskit, including dense-angle encoding [1] and IQP encoding [2], which were identified as underrepresented in applied tooling.

3. **A natural language explanation layer** that generates structured justifications in Portuguese and English, citing concrete metrics (number of qubits, circuit depth, Kernel-Target Alignment) derived from actual simulation results.

4. **A quantum kernel evaluation endpoint** that computes $K[i,j] = |\langle\phi(x_i)|\phi(x_j)\rangle|^2$ for a dataset, returns KTA scores, and generates a heatmap visualization — enabling encoding quality assessment before any classifier training.

5. **A multi-agent deployment architecture** on Red Hat OpenShift with NVIDIA OpenShell security sandboxes, exposing three OpenClaw agents with distinct communicative personalities for expert and novice audiences.

6. **A Kubeflow Pipeline** for Red Hat OpenShift AI that orchestrates the full workflow — data analysis, encoding recommendation, comparison, kernel evaluation, and MLflow artifact logging — as a reproducible five-stage DAG.

The system is released as open source at https://github.com/anapaulaappel/quantum-encoding-agents, with agent configuration at https://github.com/anapaulaappel/openclaw-quantum-agents. All code, policies, agent configurations, and the Kubeflow pipeline were developed and are maintained by the author.

2. Related Work

2.1 Quantum Data Encoding Strategies

The theoretical landscape of quantum data encoding has been shaped by a sequence of foundational papers. Schuld and Killoran [4] established the connection between quantum feature maps and kernel methods, showing that quantum circuits define kernels on data through the inner product of encoded states in Hilbert space. This framing — encoding as an implicit kernel — is the basis for all kernel-based QML methods.

Havlíček et al. [2] operationalized this idea with a concrete circuit construction: the ZZ feature map, which applies Hadamard layers followed by diagonal Pauli rotations encoding both linear (x_i^2) and cross-product ($x_i \cdot x_j$) terms. This circuit, equivalent to what we term IQP (Instantaneous Quantum Polynomial) encoding, was demonstrated on IBM quantum hardware and showed experimental advantage over classical SVMs on a constructed classification task. Their work established the standard for quantum kernel methods and motivated subsequent analysis of which encodings produce kernels useful for classification.

Pérez-Salinas et al. [3] introduced data re-uploading, departing from the one-shot encoding paradigm. In their framework, classical data is inserted into the circuit multiple times across layers, interleaved with trainable parameters. This dramatically increases the expressibility of the encoded function class at the cost of circuit depth, and connects quantum encoding to the universal approximation theory of quantum circuits. Data re-uploading has become the de facto encoding for variational quantum classifiers and quantum neural networks.

LaRose and Coyle [6] conducted the first systematic comparative study of encoding strategies across multiple datasets and noise conditions, establishing that no single encoding dominates and that the optimal choice is dataset-dependent. Their work motivates the need for an automated selection mechanism — a gap this system addresses.

The most comprehensive treatment is Sammartino's 2026 survey [1], which reviews 66 papers from 2017 to 2026, classifies encodings into six families, and derives practical selection guidelines. Crucially, Sammartino formalizes the critical gate error threshold: for hardware with gate error rate $p \geq p^* \approx 10^{-3}$, deep encodings (amplitude, IQP, custom feature map) accumulate noise faster than their expressibility advantage compensates, making shallow encodings (angle, dense angle, data re-uploading) preferable. Dense-angle encoding — which packs two features per qubit via $R_y(x_{2i}) \cdot R_z(x_{2i+1})$ — is identified as the most underused encoding family in applied QML despite its favorable depth-qubit tradeoff.

2.2 Encoding as an Implicit Kernel

A result fundamental to the theoretical grounding of this system is Schuld's 2021 theorem [S21]: every supervised quantum model that encodes data into a quantum state $|\phi(x)\rangle$ and measures an observable is mathematically equivalent to a kernel support vector machine with kernel:

$$K(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$$

This equivalence has a decisive practical implication stated explicitly in [S21]: *"the way that data is encoded into quantum states is the main ingredient that can potentially set quantum models apart from classical machine learning models."* Furthermore, kernel-based training — which is what QSVM performs directly — is provably at least as good as variational circuit training for the same encoding. The choice of encoding is therefore not a preprocessing decision; it is the core modeling decision, equivalent to choosing a kernel in classical SVM. Different encodings define different kernels: angle encoding defines a product-of-cosines kernel over the feature space; IQP encoding defines a kernel that includes cross-feature interaction terms $\cos(x_i x_j)$; custom feature map encoding defines an entangled kernel over the full feature Hilbert space. This system makes this choice explicit and measurable — the Kernel-Target Alignment (KTA) score computed by the `/v1/kernel` endpoint is a direct, label-aware evaluation of the implicit kernel defined by any encoding on the actual data.

2.3 Trainability and Barren Plateaus

The trainability of quantum circuits is a central concern for encoding choices used in variational algorithms. McClean et al. [5a] demonstrated that randomly initialized quantum circuits exhibit barren plateaus: exponentially vanishing gradients that make training infeasible beyond moderate qubit counts. This finding constrains the choice of encoding for variational methods — encodings that produce highly entangled states may exacerbate barren plateaus. Cerezo et al. [5b] surveyed the landscape of variational quantum algorithms, including the encoding strategies that couple best with trainable ansatze, establishing data re-uploading as the preferred input encoding for QNNs and VQCs due to its per-layer data injection that maintains gradient flow.

2.4 Expressibility, Entanglement, and the Encoding Tradeoff

The encoding selection problem has a fundamental three-way tension that the article must make explicit. Sim, Johnson and Aspuru-Guzik [Sim19] define *expressibility* as the deviation of a parameterized quantum circuit's output distribution from the Haar measure — circuits that cover Hilbert space densely have high expressibility. Their key result: shallow circuits have low expressibility and are confined to structured submanifolds of the state space, which may be insufficient to separate classes. This is the complementary risk to the barren plateau: too-shallow encodings underfit in Hilbert space, while too-deep circuits become untrainable due to vanishing gradients [5a]. Sammartino [1] adds the third axis: above $p^* \approx 10^{-3}$, the depth budget is set by hardware decoherence, not by expressibility requirements. The encoding recommendation policy in the system navigates all three axes simultaneously — the base heuristic selects the encoding with sufficient expressibility for the data profile, hardware refinement caps depth to the feasible regime, and KTA provides empirical evidence that the chosen encoding actually aligns with the classification task.

Larocca et al. [Lar23] further establish that over-parameterized quantum neural networks undergo a phase transition from a barren plateau regime to a trainable regime as the number of parameters

exceeds a critical threshold, directly linking encoding circuit depth to parameter count and trainability. This connects to the system's recommendation logic: data re-uploading encoding, which inserts the data in each layer rather than once, effectively increases the parameter density per qubit and maintains gradient flow, making it the preferred encoding for variational methods.

2.5 Quantum Advantage and Scope of NISQ-Era QML

A fundamental question any QML paper must address is whether quantum advantage is achievable in practice. The honest answer is nuanced. Huang et al. [Hua22] proved exponential quantum advantage in a specific learning task — predicting properties of quantum physical systems from quantum measurements — demonstrated on 40-qubit superconducting hardware. However, this advantage is task-specific: it applies when the data itself is quantum in origin, not when encoding classical tabular data into quantum circuits. Huang, Kueng and Preskill [Hua21] establish complementary information-theoretic bounds showing that classical ML can match quantum ML on average over input distributions, but quantum advantage is possible worst-case.

For classical datasets on NISQ hardware — the regime this system targets — quantum advantage over classical methods has not been proven in general. The encoding-based perspective offers a different motivation: quantum feature maps may define kernels over Hilbert space geometries that are classically hard to compute explicitly but easy to evaluate via quantum circuit execution [4, S21]. Whether this advantage materializes for any specific dataset is precisely what KTA measurement enables to assess *before* training — which is the contribution of the `/v1/kernel` endpoint. Bowles, Ahmed and Schuld [Bow24] further show, in a large-scale study across 12 QML models and 160 datasets, that classical baselines frequently match or outperform quantum classifiers at current qubit scales, underscoring that encoding evaluation is not optional but essential. This system is therefore designed as a NISQ-era engineering tool for informed encoding selection, not as a claim of general quantum supremacy.

2.6 NISQ vs. Fault-Tolerant Quantum Computing

The hardware_profile constraints in this system encode NISQ-era realities that will shift significantly in the fault-tolerant regime. On current NISQ devices — characterized by gate error rates of 10^{-3} to 10^{-2}, qubit counts of 10–400, and no error correction — circuit depth is the primary engineering constraint. Deep encodings such as amplitude encoding (depth $O(2^n)$) and multi-layer custom feature maps become impractical above the p^* threshold. In the fault-tolerant era, with logical qubits suppressing error to arbitrarily low levels, this constraint disappears: amplitude encoding's exponential qubit compression becomes viable, and the encoding choice is driven purely by expressibility and task alignment rather than hardware feasibility. The NISQ-specific recommendation logic is explicitly scoped: the `hardware_profile` field allows users to specify their hardware regime, and the system's behavior changes accordingly — a user passing `gate_error_rate: 0.0` (perfect hardware, simulation) receives recommendations based on expressibility alone, while a user passing `gate_error_rate: 5e-3` (IBM Eagle) receives hardware-adjusted recommendations that deprioritize deep encodings. This design is forward-compatible: as hardware improves, the same

system surfaces richer encodings for the same data.

2.7 AI Agents for Scientific Tooling

The use of LLM-based agents as interfaces for scientific computing tools is an emerging paradigm. Function-calling LLMs [Meta AI, 2024; OpenAI, 2023] allow agents to invoke specialized backends — simulators, optimizers, databases — based on natural language intent. This approach has been applied to chemistry [Boiko et al., 2023], materials science, and bioinformatics, but has not been systematically applied to quantum computing. To my knowledge, this is the first system to deploy multi-personality LLM agents specifically for quantum encoding recommendation.

The OpenClaw agent framework [OpenClaw, 2026] provides the infrastructure for agent deployment with persistent memory (via `MEMORY.md`), user calibration (via `BOOTSTRAP.md` and `USER.md`), and skill encapsulation. The skills system allows shared behavioral modules (`$qiskit-api`, `$circuit-review`) to be loaded on demand across agents with different communicative styles.

2.8 Secure Agent Deployment

The security of autonomous agents operating on computational infrastructure is an open problem. NVIDIA OpenShell [NVIDIA, 2026] addresses this through out-of-process policy enforcement: each agent runs in an isolated sandbox governed by a YAML policy that constrains filesystem access, network endpoints (at HTTP method granularity, enforced by Landlock LSM + seccomp), and process privileges. The policy engine validates rules using the Z3 SMT solver before application, providing formal correctness guarantees unavailable in traditional Kubernetes NetworkPolicy. In this deployment, each agent receives a distinct policy: the expert agent (Circuit) is restricted to the Qiskit API and LLM endpoints only; the mentor agent (Quanta) additionally has read-only access to arXiv and Qiskit documentation, reflecting the pedagogical needs of its communicative role.

---

3. Problem Description

3.1 The QML Training Pipeline

To position the encoding selection problem, I first establish the full QML training loop in which it occurs. A supervised quantum machine learning pipeline consists of five stages executed iteratively:

```
Classical data x


[1] Data Encoding          U(x)|0  → | (x)         ← THIS PAPER


[2] Parameterized Ansatz   V( )| (x)  → | (x, )    ← trainable circuit
```

```
[3] Measurement              (x, )|M| (x, )  → ŷ   ← expectation value

[4] Classical Loss           L(ŷ, y) → scalar

[5] Classical Optimizer      ←  -  L              ← gradient descent

                             repeat
```

The encoding layer [1] is fixed for the duration of training — it is a hyperparameter of the model, not a trainable parameter. This makes the encoding choice more consequential than any single trainable parameter: a poorly chosen encoding defines a kernel that cannot linearly separate the classes in Hilbert space, no matter how many training iterations the optimizer runs. The ansatz [2] amplifies or suppresses the structure introduced by the encoding but cannot recover structure that was not encoded. Stage [3] collapses the quantum state to a classical scalar through measurement; the encoding determines how much of the data's structure survives this collapse. This system operates at stage [1], with KTA providing a measurement of whether the chosen encoding creates a Hilbert space geometry useful for the loss at stage [4].

For quantum kernel methods (QSVM), stages [2]–[5] are replaced by classical SVM training on the kernel matrix $K[i,j] = |\langle\phi(x_i)|\phi(x_j)\rangle|^2$ — which is exactly what the `/v1/kernel` endpoint computes. In this case, the encoding is the *entire* model specification, reinforcing the centrality of stage [1].

3.2 The Encoding Selection Problem

Let $\mathcal{D} = \{x_i\}_{i=1}^N \subset \mathbb{R}^d$ be a classical dataset with N samples and d features. A quantum encoding is a parameterized map $\phi : \mathbb{R}^d \to \mathcal{H}$ from the feature space to a Hilbert space $\mathcal{H}$ of dimension 2^n, implemented as a quantum circuit $U(x)$ acting on n qubits such that $|\phi(x)\rangle = U(x)|0\rangle^{\otimes n}$.

The encoding selection problem is: given $\mathcal{D}$ and an optional QML task specification $\mathcal{T}$ (classification, kernel method, variational circuit, etc.) and hardware constraints $\mathcal{H}w$ (gate error rate, qubit count, connectivity topology), select an encoding ϕ^* that maximizes downstream task performance subject to hardware feasibility.

This problem is ill-posed for two reasons. First, the downstream task performance depends on the full learning pipeline, not just the encoding; it cannot be evaluated without training a model. Second, the interaction between data characteristics and encoding properties is non-linear and high-dimensional: the same encoding that works well on dense continuous data may fail on binary or categorical data, and the same circuit that is feasible on a simulator may degrade catastrophically on hardware above the p^* threshold.

3.3 Current Practice and Its Limitations

In practice, encoding selection is rarely principled. A survey of QML implementations in the literature reveals that the majority use angle encoding as a default, regardless of dataset structure. The primary reasons are familiarity and simplicity: one Ry rotation per feature produces a shallow, interpretable circuit. However, this choice ignores:

- **Dimensionality**: for datasets with $d > 10$ features, angle encoding requires d qubits, which may exceed hardware constraints. Dense-angle encoding achieves the same depth with $\lceil d/2 \rceil$ qubits.
- **Task alignment**: for quantum kernel methods (QSVM), encodings without entanglement (angle, dense-angle) define kernels with limited expressibility in Hilbert space. IQP and custom feature map encodings, which generate cross-feature correlations via two-qubit gates, define richer kernels.
- **Hardware noise**: above $p^* \approx 10^{-3}$, amplitude encoding's deep state preparation circuit accumulates noise faster than its qubit compression advantage is worth. On IBM Eagle hardware ($p \approx 5 \times 10^{-3}$), the effective circuit depth after error accumulation can reduce the signal-to-noise ratio below classification threshold.
- **Data type**: basis encoding, the natural choice for binary or categorical data, is systematically ignored when practitioners default to angle encoding for all data types.

3.4 The Accessibility Gap

Beyond the technical selection problem, there is an accessibility gap: the knowledge required to make principled encoding choices — distributed across a dozen papers published between 2018 and 2026, requiring familiarity with quantum information theory, circuit complexity, and NISQ hardware characteristics — is not accessible to the ML practitioner approaching QML for the first time, nor to the domain expert (chemist, biologist, financial analyst) who wants to apply QML without deep quantum expertise.

This gap manifests in two distinct user profiles:

The QML practitioner (expert) needs dense, metric-first information: encoding name, circuit depth, qubit count, KTA score, and a reference to the relevant paper. They do not need explanations of what a qubit is.

The ML practitioner approaching QML (learner) needs progressive disclosure: physical intuition before mathematics, the Bloch sphere before Dirac notation, a concrete analogy before a formal definition. They need to understand *why* the recommendation makes sense before trusting it.

Current tools address neither profile. The Qiskit documentation provides reference material but no recommendation engine. The academic literature provides theoretical frameworks but no implementation. The system I present bridges this gap through specialized agents calibrated to each

profile.

---

4. Quantum Encoding Agents: System Design

4.1 Architecture Overview

The system consists of three loosely coupled layers (Figure 1):

```
User (natural language, any channel)

   OpenClaw Gateway          ← agent personality via SOUL.md + AGENTS.md
   LLM (Qwen2.5-Coder-14B   ← served via Ollama (local) or vLLM (OpenShift)
        or Llama-3.2-3B)

           POST /v1/recommend/explain

             llama-qiskit-agents (FastAPI)
                      infer_data_profile()        → DataProfile
                      recommend_encoding()        → EncodingType + reason
                         refine_recommendation() → hardware-aware adjustment
                      detect_language()           → pt | en
                      build_natural_explanation() → structured narrative
                      generate_qiskit_code()      → complete Python code
                      simulate_encoding_circuit() → AerSimulator (CPU)
                      render_bloch_sphere()       → PNG base64
                      compute_kernel()            → K[i,j] + KTA + heatmap
```

The microservice (`llama-qiskit-agents`) is a stateless FastAPI application that can be deployed independently of any agent framework and called directly via HTTP. The agent layer (OpenClaw) adds conversational memory, user calibration, and multi-turn interaction. The security layer (OpenShell) enforces per-agent network and filesystem policies at the OS level, outside the application.

4.2 Data Profile Analysis

The first processing step is structured data profiling. Given input in any of three forms — a numpy array, a CSV file, or a natural language description — the system extracts a `DataProfile` dataclass:

```python
@dataclass
class DataProfile:
    n_samples: int
    n_features: int
    is_binary: bool        # all values  {0, 1}
    is_categorical: bool   #  20 unique values, not continuous
    is_continuous: bool    # >10 unique values or float dtype
```

```
    has_negative: bool      # any value < -1e-9
    description: str
```

For text descriptions, keyword detection in both Portuguese and English maps phrases to profile flags (e.g., "binary", "binário", "bit" → `is_binary = True`). This enables natural language input without requiring the user to provide numerical data.

4.3 The Seven Encoding Strategies

The system implements seven encoding families as Qiskit `QuantumCircuit` objects. Table 1 summarizes their properties.

Table 1. Quantum encoding strategies implemented in the system.

Encoding	Qubits	Depth	Recommended for	Key Gates
Amplitude	$\lceil \log_2 n \rceil$	$O(2^n)$	Large vectors, qubit-constrained	`StatePreparation`
Angle	d	1	$d \leq 4$, continuous, prototyping	$R_y(x_i)$
Dense Angle [1]	$\lceil d/2 \rceil$	2	$5 \leq d \leq 12$, continuous, no negatives	$R_y(x_{2i}) \cdot R_z(x_{2i+1})$
IQP [2]	d	$\sim 3d$	$8 \leq d \leq 16$, quantum kernels	$H + R_z(x_i^2) + R_{zz}(x_i x_j)$
Basis	d	$\leq d$	Binary/categorical data	X per 1-bit
Data Re-uploading [3]	d	$\sim 4d$	VQC, QNN, variational	$R_y \times L + CX$ chain
Custom Feature Map	d	$\sim 3d$	QSVM, arbitrary kernels	$H + R_z + R_y + CZ$ pairwise

Dense-angle encoding [1] is the key addition motivated by the survey. It achieves half the qubit count of standard angle encoding at depth 2 by packing two features per qubit:

$$U_{DA}(x)|0\rangle^{\otimes \lceil d/2 \rceil} = \bigotimes_{i=0}^{\lceil d/2 \rceil - 1} R_z(x_{2i+1}) R_y(x_{2i}) |0\rangle_i$$

IQP encoding [2] is implemented without `qiskit-machine-learning` dependency through manual decomposition of $R_{zz}(\theta)$ as $CX \cdot R_z(\theta) \cdot CX$:

$$U_{IQP}(x) = H^{\otimes d} \cdot \prod_i R_z(x_i^2) \cdot \prod_{i<j} R_{zz}(x_i x_j) \cdot H^{\otimes d}$$

Only adjacent pairs $(i, i+1)$ are connected, keeping circuit depth polynomial in d and compatible with linear and heavy-hex hardware topologies.

4.4 Hardware-Aware Recommendation Policy

The recommendation pipeline has two stages: data-driven base selection and hardware-constrained refinement.

Base selection applies a priority decision tree over the `DataProfile`:

1. Binary, $d \leq 16 \rightarrow$ `basis`
2. Continuous, $d \leq 4 \rightarrow$ `angle`
3. Continuous, $4 < d \leq 12$, no negatives $\rightarrow$ `dense_angle`
4. Single sample, $d \geq 4$, continuous $\rightarrow$ `amplitude`
5. Continuous, $8 < d \leq 16 \rightarrow$ `iqp`
6. Continuous, $d > 16 \rightarrow$ `custom_feature_map`
7. Default $\rightarrow$ `angle`

Task refinement overrides the base selection based on the QML task: kernel methods and QSVM always elevate to `custom_feature_map`; variational methods (VQC, QNN) prefer `data_reuploading`; binary classification with binary data uses `basis`.

Hardware refinement applies the p^* threshold from Sammartino [1]. For any `HardwareProfile` with `gate_error_rate >= 1e-3`, deep encodings (`amplitude`, `custom_feature_map`, `iqp`) are replaced by the shallowest continuous alternative:

```
if hw.gate_error_rate >= p* and enc in DEEP_ENCODINGS:
    enc = dense_angle if d > 4 else angle
    explanation += "Hardware adjustment: gate_error_rate={:.1e}  p*..."
```

Additionally, `connectivity: heavy-hex` or `linear` triggers a SWAP overhead warning for `custom_feature_map` (which requires pairwise CZ gates), and `max_depth_budget` triggers a depth feasibility warning with estimated depth per encoding.

4.5 Natural Language Explanation Generation

The explanation layer generates a four-paragraph structured narrative in the detected language, each paragraph citing concrete values from the `DataProfile` and `SimulationResult`:

1. **Data description**: number of samples, features, value type, sign.
2. **Encoding justification**: why this encoding family fits the data — geometric intuition, qubit count, circuit depth with NISQ compatibility assessment.

3. **Circuit metrics**: "The simulated `dense_angle` circuit uses **3 qubits** and has **depth 2**. Very low depth: compatible with current NISQ hardware without error correction."
4. **Comparative analysis**: why each alternative encoding is less suitable, citing concrete depth and qubit differences.

Language detection uses vocabulary overlap between the input text and Portuguese/English marker sets. The same explanation function handles both languages through parallel string dictionaries indexed by `EncodingType`.

4.6 Quantum Kernel Evaluation

For datasets where classification labels are available, the system computes the quantum kernel matrix $K \in [0,1]^{N \times N}$:

$$K[i,j] = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$$

Statevectors are computed using the `StatevectorSimulator` (exact, without measurement shots) and inner products are computed classically. The Kernel-Target Alignment [Cortes et al., 2012] is computed as:

$$\text{KTA}(K, y) = \frac{\langle K, yy^\top \rangle_F}{\|K\|_F \cdot \|yy^\top\|_F}$$

where $y \in \{-1, +1\}^N$ is the label vector. KTA provides a label-aware measure of how well the encoding separates classes in Hilbert space, independently of any trained classifier. The result is returned as a JSON matrix, a heatmap visualization (viridis colormap, per-class tick coloring), and a natural language caption interpreting the KTA score.

In a preliminary evaluation on a synthetic two-class dataset ($N = 4$, $d = 2$), angle encoding achieved KTA = 0.701 while custom feature map achieved KTA = 0.114 — demonstrating that richer encodings do not necessarily produce better-aligned kernels, and that empirical evaluation via KTA is essential before committing to a training pipeline.

4.7 Bloch Sphere Visualization

For encodings with $n \leq 6$ qubits, the system optionally captures the statevector before measurement and renders the Bloch sphere representation of each qubit's marginal state using `qiskit.visualization.plot_bloch_multivector`. The resulting PNG (base64-encoded) accompanies the recommendation when `include_bloch: true` is set in the request. This provides a geometric intuition for what the encoding does to each data point that no numerical summary can convey.

4.8 Agent Personalities and the Role of SOUL.md

Three OpenClaw agents expose the recommendation system through distinct communicative registers, calibrated for different user expertise levels.

QiskitAgent (`quantum_encoding`) is the general-purpose agent: it recommends encodings, generates code, and presents results without assumptions about the user's prior knowledge.

Circuit (`qiskit_expert`) is calibrated for QML practitioners. Its `SOUL.md` instructs it to presuppose familiarity with key concepts, respond in three dense blocks (encoding name, metrics, rationale), never explain superposition, present trade-offs as two objective sides without choosing, and request missing information (n_features, hardware target, depth budget) when the query is underspecified. Circuit cites papers from the literature when relevant without prompting.

Quanta (`qiskit_mentor`) is calibrated for learners approaching QML from classical ML or physics backgrounds. Its `SOUL.md` prohibits the Schrödinger's cat analogy (which lies) and mandates physical intuition before mathematics — the Bloch sphere before the Dirac notation. Quanta structures responses in four named sections, calibrates explanation depth from conversational signals, and records in `MEMORY.md` which analogies worked and which concepts caused confusion, updating its approach across sessions.

Both agents perform a calibration ritual on first session (`BOOTSTRAP.md`) that asks three targeted questions to populate `USER.md` with a persistent user profile (hardware target, QML focus, experience level, preferred language). Subsequent sessions inject `USER.md` automatically, allowing the agent to skip explanations of already-mastered concepts.

4.9 Skill Architecture

Two shared skills encapsulate reusable behavior across all agents:

`$qiskit-api` documents the `/v1/recommend/explain` endpoint — request fields, response fields, and rules of presentation — as a `SKILL.md` file. When an agent invokes this skill, it receives a compact, authoritative specification of how to call the API and how to present each response field. This eliminates documentation duplication across agents and ensures consistent behavior when the API evolves.

`$circuit-review` provides a four-step protocol for reviewing an existing circuit pasted by the user: identify the encoding, extract metrics, call `/v1/compare` for benchmark, and issue one of three verdicts (keep / optimize / replace) with a technical rationale.

4.10 Security Architecture (OpenShell)

Each agent runs inside a dedicated NVIDIA OpenShell sandbox governed by a YAML policy validated by the Z3 SMT solver. The policies enforce least-privilege at HTTP method granularity:

- **Circuit** (expert): access limited to `qiskit-api:8080` (read-write) and the LLM endpoint (POST only). No external internet access.

- **Quanta** (mentor): same as Circuit, plus read-only access to `arxiv.org` and `docs.quantum.ibm.com` — reflecting Quanta's need to cite academic references and official documentation.
- **QiskitAgent** (general): same network scope as Circuit, with write access to `/tmp` for CSV uploads via `/v1/compare/csv`.

The `process` block sets `run_as_user: sandbox`. Network policies use `access: read-only` and `access: read-write` presets (correct OpenShell schema; the non-existent `allowed_methods` / `allowed_paths` fields are not used). Binaries are specified as objects with `path` keys, enabling per-binary policy enforcement through Landlock LSM.

4.11 Kubeflow Pipeline for RHOAI

The full workflow is codified as a Kubeflow Pipelines v2 DAG deployable on Red Hat OpenShift AI:

```
CSV input + QML parameters
  → [1] analyze_data      (POST /v1/analyze)
  → [2] recommend_encoding (POST /v1/recommend/explain)
  → [3] compare_encodings  (POST /v1/compare)
  → [4] compute_kernel     (POST /v1/kernel, when labels provided)
  → [5] log_to_mlflow      (params + metrics + heatmap + Bloch sphere + code)
```

Each step is an independent KFP v2 component with CPU/memory resource limits. The pipeline accepts hardware profile parameters (`gate_error_rate`, `connectivity`, `max_depth_budget`) that propagate to the recommendation engine, enabling reproducible NISQ-aware experiments. Artifacts logged to MLflow include the explanation text, the generated Qiskit code, the kernel heatmap PNG, and the Bloch sphere visualization.

---

5. Discussion

5.1 The Role of KTA in Practice

My preliminary results suggest that KTA is a more reliable pre-training signal than circuit expressibility alone. Angle encoding (KTA = 0.701) outperformed the custom feature map (KTA = 0.114) on the test dataset despite the latter's greater theoretical richness. This confirms LaRose and Coyle's finding [6] that no encoding universally dominates, and motivates the design approach: rather than recommending based on theoretical properties alone, I compute KTA on the actual data and present the score alongside the recommendation.

5.2 Limits of Text-Based Encoding Selection

The natural language input path — inferring data profile from a description — has inherent limitations. Keyword detection can misclassify ambiguous descriptions ("data with real-valued features"

simultaneously matches `is_continuous` and could match `has_negative`). For production use, numerical data or CSV upload should be preferred. The text path is primarily useful for initial exploration and for the conversational interface.

5.3 Agent Personality as an Engineering Artifact

The `SOUL.md`-based personality specification is a form of behavioral engineering that sits between prompt engineering and agent architecture. Unlike a system prompt, `SOUL.md` is persistent (injected every session), versioned in git, and testable: a response from Circuit that begins with "Great question!" violates the soul specification and can be detected in evaluation. This framing — soul as contract — enables more systematic quality assurance of agent behavior than ad-hoc prompting.

5.4 OpenShell Policy Design as Security-by-Data-Profile

The assignment of different network permissions to different agents based on their communicative role is a novel security design pattern: **policy as a function of agent persona**. Quanta requires external internet access (arXiv, Qiskit docs) because its role is pedagogical and citation-heavy. Circuit does not, because experts are expected to know the literature. This reflects the insight that the minimum necessary privilege of an AI agent is determined not just by its technical function but by its communicative purpose.

---

6. Conclusion

I presented Quantum Encoding Agents, a system that addresses the encoding selection problem in Quantum Machine Learning through a combination of hardware-aware recommendation logic, natural language generation, quantum kernel evaluation, and specialized AI agents calibrated to different user expertise levels. The system is grounded in the theoretical framework of Sammartino [1] and Havlíček et al. [2], implements seven encoding families including the underused dense-angle encoding, and provides the first open-source implementation of KTA-based encoding quality assessment via a REST API.

The multi-agent architecture demonstrates that AI agents deployed for scientific tools can and should have distinct communicative personalities calibrated to their user populations — not as a cosmetic feature, but as a substantive design choice that determines what information to surface, in what order, and with what degree of assumed prior knowledge. The security architecture extends this idea: agent policy is a function of agent role, not just of agent function.

Future work includes: (i) integration with IBM Quantum real hardware via `QiskitRuntimeService` for empirical NISQ validation; (ii) the Q-Tucker shallow state preparation [arXiv:2602.09909] as an alternative amplitude encoding with $O(n)$ depth; (iii) the Quantum Spectral Model [arXiv:2607.22516] as an adaptive encoding whose frequency spectrum is conditioned on the input; and (iv) a KTA-guided encoding search that evaluates multiple candidates and returns the one with the highest alignment score.

The system is available at: - API: https://github.com/anapaulaappel/quantum-encoding-agents - Agents: https://github.com/anapaulaappel/openclaw-quantum-agents

---

References

[1] V. Sammartino, "Feature Encoding in Quantum Machine Learning: A Survey and Practical Guidelines," *arXiv preprint*, arXiv:2606.05387 [quant-ph], Jun. 2026. doi: 10.48550/arXiv.2606.05387

[2] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, Mar. 2019. doi: 10.1038/s41586-019-0980-2

[3] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. I. Latorre, "Data re-uploading for a universal quantum classifier," *Quantum*, vol. 4, p. 226, Feb. 2020. doi: 10.22331/q-2020-02-06-226

[4] M. Schuld and N. Killoran, "Quantum machine learning in feature Hilbert spaces," *Physical Review Letters*, vol. 122, no. 4, p. 040504, Feb. 2019. doi: 10.1103/PhysRevLett.122.040504

[5a] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, "Barren plateaus in quantum neural network training landscapes," *Nature Communications*, vol. 9, Art. no. 4812, Nov. 2018. doi: 10.1038/s41467-018-07090-4

[5b] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, "Variational Quantum Algorithms," *Nature Reviews Physics*, vol. 3, pp. 625–644, 2021. doi: 10.1038/s42254-021-00348-9

[6] R. LaRose and B. Coyle, "Robust data encodings for quantum classifiers," *Physical Review A*, vol. 102, no. 3, p. 032420, Sep. 2020. doi: 10.1103/PhysRevA.102.032420

[7] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," *arXiv preprint*, arXiv:2405.08810, 2024. doi: 10.48550/arXiv.2405.08810

[8] M. E. Sahin, E. Altamura, O. Wallis, S. P. Wood, A. Dekusar, D. A. Millar, T. Imamichi, A. Matsuo, and S. Mensa, "Qiskit Machine Learning: an open-source library for quantum machine learning tasks at scale on quantum hardware and classical simulators," *arXiv preprint*, arXiv:2505.17756, May 2025. doi: 10.48550/arXiv.2505.17756

[9] Kubeflow Project, "Kubeflow Pipelines," The Linux Foundation, 2026. [Online]. Available: https://www.kubeflow.org/docs/components/pipelines/

[10] Red Hat, Inc., "Red Hat OpenShift AI," 2026. [Online]. Available: https://www.redhat.com/en/products/ai/o
ai

[11] Ollama Inc., *Ollama*, open-source software, MIT License, 2026. [Online]. Available: https://ollama.com / https://github.com/ollama/ollama

[12] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

[13] OpenClaw, *OpenClaw AI Assistant*, 2026. [Online]. Available: https://docs.openclaw.ai

[14] NVIDIA Corporation, "NVIDIA NIM on Red Hat OpenShift AI," 2026. [Online]. Available: https://developer.nvidia.com/nim

[S21] M. Schuld, "Supervised quantum machine learning models are kernel methods," *arXiv preprint*, arXiv:2101.11020, 2021. doi: 10.48550/arXiv.2101.11020

[Sim19] S. Sim, P. D. Johnson, and A. Aspuru-Guzik, "Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms," *Advanced Quantum Technologies*, vol. 2, no. 12, p. 1900070, 2019. doi: 10.1002/qute.201900070

[Hua22] H.-Y. Huang, R. Kueng, G. Torlai, V. V. Albert, and J. Preskill, "Quantum advantage in learning from experiments," *Science*, vol. 376, no. 6598, pp. 1182–1186, Jun. 2022. doi: 10.1126/science.abn7293

[Hua21] H.-Y. Huang, R. Kueng, and J. Preskill, "Information-theoretic bounds on quantum advantage in machine learning," *Physical Review Letters*, vol. 126, no. 19, p. 190505, May 2021. doi: 10.1103/PhysRevLett.126.190505

[Bow24] J. Bowles, S. Ahmed, and M. Schuld, "Better than classical? The subtle art of benchmarking quantum machine learning models," *arXiv preprint*, arXiv:2403.07059, 2024. doi: 10.48550/arXiv.2403.07059

[Lar23] M. Larocca, F. Sauvage, F. M. Sbahi, G. Verdon, P. J. Coles, and M. Cerezo, "Group-invariant quantum machine learning," *Nature Computational Science*, vol. 3, pp. 542–551, 2023. doi: 10.1038/s43588-023-00467-6

Appendix A: BibTeX Entries

```
@misc{sammartino2026,
  title         = {Feature Encoding in Quantum Machine Learning:
                   A Survey and Practical Guidelines},
  author        = {Sammartino, Vincenzo},
  year          = {2026},
  eprint        = {2606.05387},
  archivePrefix = {arXiv},
  primaryClass  = {quant-ph},
  doi           = {10.48550/arXiv.2606.05387}
}
```

```
@article{havlicek2019,
  author  = {Havl{\'\i}{\v{c}}ek, Vojt{\v{e}}ch and C{\'o}rcoles, Antonio D.
             and Temme, Kristan and Harrow, Aram W. and Kandala, Abhinav
             and Chow, Jerry M. and Gambetta, Jay M.},
  title   = {Supervised learning with quantum-enhanced feature spaces},
  journal = {Nature},
  volume  = {567},
  number  = {7747},
  pages   = {209--212},
  year    = {2019},
  doi     = {10.1038/s41586-019-0980-2}
}

@article{perezsalinas2020,
  author  = {P{\'e}rez-Salinas, Adri{\'a}n and Cervera-Lierta, Alba
             and Gil-Fuster, Elies and Latorre, Jos{\'e} I.},
  title   = {Data re-uploading for a universal quantum classifier},
  journal = {Quantum},
  volume  = {4},
  pages   = {226},
  year    = {2020},
  doi     = {10.22331/q-2020-02-06-226}
}

@article{schuld2019,
  author  = {Schuld, Maria and Killoran, Nathan},
  title   = {Quantum machine learning in feature {H}ilbert spaces},
  journal = {Physical Review Letters},
  volume  = {122},
  number  = {4},
  pages   = {040504},
  year    = {2019},
  doi     = {10.1103/PhysRevLett.122.040504}
}

@article{mcclean2018,
  author  = {McClean, Jarrod R. and Boixo, Sergio and Smelyanskiy,
             Vadim N. and Babbush, Ryan and Neven, Hartmut},
  title   = {Barren plateaus in quantum neural network training landscapes},
  journal = {Nature Communications},
  volume  = {9},
  pages   = {4812},
```

```
  year    = {2018},
  doi     = {10.1038/s41467-018-07090-4}
}

@article{cerezo2021,
  author  = {Cerezo, M. and Arrasmith, Andrew and Babbush, Ryan and
             Benjamin, Simon C. and Endo, Suguru and Fujii, Keisuke and
             McClean, Jarrod R. and Mitarai, Kosuke and Yuan, Xiao and
             Cincio, Lukasz and Coles, Patrick J.},
  title   = {Variational {Q}uantum {A}lgorithms},
  journal = {Nature Reviews Physics},
  volume  = {3},
  pages   = {625--644},
  year    = {2021},
  doi     = {10.1038/s42254-021-00348-9}
}

@article{larose2020,
  author  = {LaRose, Ryan and Coyle, Brian},
  title   = {Robust data encodings for quantum classifiers},
  journal = {Physical Review A},
  volume  = {102},
  number  = {3},
  pages   = {032420},
  year    = {2020},
  doi     = {10.1103/PhysRevA.102.032420}
}

@misc{qiskit2024,
  title         = {Quantum computing with {Q}iskit},
  author        = {Javadi-Abhari, Ali and Treinish, Matthew and Krsulich, Kevin
                   and Wood, Christopher J. and Lishman, Jake and Gacon, Julien
                   and Martiel, Simon and Nation, Paul D. and Bishop, Lev S.
                   and Cross, Andrew W. and Johnson, Blake R. and Gambetta, Jay M.},
  year          = {2024},
  eprint        = {2405.08810},
  archivePrefix = {arXiv},
  primaryClass  = {quant-ph},
  doi           = {10.48550/arXiv.2405.08810}
}

@misc{qiskit_ml2025,
```

```
  author        = {Sahin, M. Emre and Altamura, Edoardo and Wallis, Oscar
                   and Wood, Stephen P. and Dekusar, Anton and Millar, Declan A.
                   and Imamichi, Takashi and Matsuo, Atsushi and Mensa, Stefano},
  title         = {{Qiskit Machine Learning}: an open-source library for quantum
                   machine learning tasks at scale},
  year          = {2025},
  eprint        = {2505.17756},
  archivePrefix = {arXiv},
  primaryClass  = {quant-ph},
  doi           = {10.48550/arXiv.2505.17756}
}

@article{zaharia2018,
  author  = {Zaharia, Matei and Chen, Andrew and Davidson, Aaron and Ghodsi, Ali
             and Hong, Sue Ann and Konwinski, Andy and Murching, Siddharth and
             Nykodym, Tomas and Ogilvie, Paul and Parkhe, Mani and Xie, Fen
             and Zumar, Corey},
  title   = {Accelerating the Machine Learning Lifecycle with {MLflow}},
  journal = {IEEE Data Engineering Bulletin},
  volume  = {41},
  number  = {4},
  pages   = {39--45},
  year    = {2018}
}

@misc{schuld2021_kernel,
  author        = {Schuld, Maria},
  title         = {Supervised quantum machine learning models are kernel methods},
  year          = {2021},
  eprint        = {2101.11020},
  archivePrefix = {arXiv},
  primaryClass  = {quant-ph},
  doi           = {10.48550/arXiv.2101.11020}
}

@article{sim2019,
  author  = {Sim, Sukin and Johnson, Peter D. and Aspuru-Guzik, Al{\'a}n},
  title   = {Expressibility and Entangling Capability of Parameterized Quantum
             Circuits for Hybrid Quantum-Classical Algorithms},
  journal = {Advanced Quantum Technologies},
  volume  = {2},
  number  = {12},
```

```
  pages   = {1900070},
  year    = {2019},
  doi     = {10.1002/qute.201900070}
}

@article{huang2022,
  author  = {Huang, Hsin-Yuan and Kueng, Richard and Torlai, Giacomo and
             Albert, Victor V. and Preskill, John},
  title   = {Quantum advantage in learning from experiments},
  journal = {Science},
  volume  = {376},
  number  = {6598},
  pages   = {1182--1186},
  year    = {2022},
  doi     = {10.1126/science.abn7293}
}

@article{huang2021,
  author  = {Huang, Hsin-Yuan and Kueng, Richard and Preskill, John},
  title   = {Information-theoretic bounds on quantum advantage in machine learning},
  journal = {Physical Review Letters},
  volume  = {126},
  number  = {19},
  pages   = {190505},
  year    = {2021},
  doi     = {10.1103/PhysRevLett.126.190505}
}

@misc{bowles2024,
  author        = {Bowles, Joseph and Ahmed, Shahnawaz and Schuld, Maria},
  title         = {Better than classical? The subtle art of benchmarking quantum
                   machine learning models},
  year          = {2024},
  eprint        = {2403.07059},
  archivePrefix = {arXiv},
  primaryClass  = {quant-ph},
  doi           = {10.48550/arXiv.2403.07059}
}

@article{larocca2023,
  author  = {Larocca, Martin and Sauvage, Fr{\'e}d{\'e}ric and Sbahi, Faris M.
             and Verdon, Guillaume and Coles, Patrick J. and Cerezo, Marco},
```

```
  title   = {Group-invariant quantum machine learning},
  journal = {Nature Computational Science},
  volume  = {3},
  pages   = {542--551},
  year    = {2023},
  doi     = {10.1038/s43588-023-00467-6}
}
```